Stage 1 — Core Menu-Only / Orderless Support

Date: September 3, 2026 **Source of truth for architecture:** `docs/reports/orderless-menu-only-pod-audit.md`

Summary

Menu-only is now a first-class product state. A platform admin can switch an individual vendor or an entire pod to menu-only. Published menus stay public and browsable, all ordering affordances disappear from the customer UI, the cart and checkout reject those lines server-side with their own error codes, and the vendor and pod dashboards stop asking for commerce setup that is not relevant. Turning ordering back on restores the previous configuration immediately — no menu republish, no reconnection of Stripe, Square, or Deliverect.

The implementation deliberately did **not** reuse any existing field to mean menu-only. Two new durable columns carry intent, and every downstream decision is derived from a single resolver rather than scattered `if (!vendor.orderingEnabled)` checks.

Two real defects were found and fixed while writing tests:

- `validateCartForOrder` (the checkout path) was reading vendor ordering intent from the caller-supplied cart payload, which meant a stale client could reach checkout for a menu-only vendor. It now reads intent from the database, like the cart-display path already did.
- `VendorMenuItemCard` treated "ordering blocked" and "item unavailable" as the same condition, which made every available item in a menu-only vendor render as if it were sold out. Those are now separate inputs.

Schema / Migration

Two additive columns:

```
model Pod {
  /// Durable product mode: when false, this is a menu-only pod — published vendor
menus stay
  /// public and browsable but no vendor can accept customer orders.
Vendor.orderingEnabled is
  /// preserved and resumes when this is turned back on. Distinct from
`mennyuOrdersPaused`
  /// (temporary intake pause) and `isActive` (public visibility).
  orderingEnabled Boolean @default(true)
}

model Vendor {
  /// Durable product mode: when false, this vendor is menu-only — its published menu
stays public
```

```
  /// and browsable but customers cannot add items to cart. Menu, menu source,
routing, and Stripe
  /// state are preserved so ordering can resume unchanged. Distinct from
`mennyuOrdersPaused`
  /// (temporary intake pause), `isActive` (public visibility), and
MenuItem.isAvailable (sold out).
  orderingEnabled Boolean @default(true)
}
```

Migration:
`prisma/migrations/20260903230000_pod_vendor_ordering_enabled/migration.sql`

```
ALTER TABLE "Pod"    ADD COLUMN IF NOT EXISTS "orderingEnabled" BOOLEAN NOT NULL
DEFAULT true;
ALTER TABLE "Vendor" ADD COLUMN IF NOT EXISTS "orderingEnabled" BOOLEAN NOT NULL
DEFAULT true;
```

Default behavior:

Property	Behavior
Existing rows	Stay orderable — `NOT NULL DEFAULT true`, no backfill
Menu data	Untouched
`menuSource` / `orderRoutingMode`	Untouched
Payment configuration	Untouched
Indexes	None added; the admin ordering filter runs inside the existing paginated vendor search, which already scans on the same predicate set
Rollback	`DROP COLUMN IF EXISTS "orderingEnabled"` on each table (documented in the migration header)

`npx prisma validate` passes.

Central State Logic

Effective orderability lives in three layered modules. Nothing else reads the raw flags to make a product decision.

`src/lib/vendor-ordering-mode.ts` (new) — owns *intent only*. It knows nothing about Stripe, POS, hours, or pause, which is what keeps menu-only from ever being confused with those states.

```
resolveVendorOrderingIntent({ podOrderingEnabled, vendorOrderingEnabled }) => {
  podOrderingEnabled, vendorOrderingEnabled, effectiveOrderingEnabled,
```

```
  menuOnly, menuOnlyByPod, menuOnlyByVendor
}
```

Undefined flags resolve to enabled, so a partial select can never accidentally mute a vendor. This module also holds every piece of menu-only copy and both error codes, so admin, customer, vendor, and pod surfaces cannot drift apart.

`src/lib/vendor-readiness-states.ts` — extended, not duplicated. It now exposes:

```
type VendorCommerceState = VendorOrderingIntent & {
  visibility: VendorPublicVisibilityState
  customerCanOrder: boolean
  orderingPrerequisitesReady: boolean   // Stripe/routing/menu, independent of intent
  blockedReason:
    | "vendor_not_public_ready" | "pod_ordering_disabled" |
"vendor_ordering_disabled"
    | "pod_orders_paused" | "vendor_paused" | "vendor_closed"
    | "ordering_setup_incomplete" | "item_unavailable" | null
  customerStatusLabel: string
  customerBannerLine: string | null
}
```

Precedence inside `getVendorOrderabilityState`: hidden → orderable → **menu-only** → pause/closed/setup. Menu-only sits above pause and setup on purpose, so a menu-only vendor with no Stripe account reports `vendor_ordering_disabled`, never "needs payment setup". In the menu-only branch, `customerBannerLine` is `null` and `showBrowseHint` is `false` — there is no outage to announce.

`src/lib/vendor-orderability-in-pod.ts` — the cart/checkout gate. Intent is resolved on *both* paths: the full readiness path and the shallow path used by cart display and quantity edits.

Public visibility is untouched by intent. `getVendorPublicVisibilityState` still depends only on active/deleted state, pod membership, public profile fields, published menu, and hours.

Admin UI

Vendor detail (`AdminVendorOverview.tsx`) — a new `Ordering mode` section, visually separate from the existing pause control, with the current state, a one-line description, and a single reason-gated action (`Switch to menu only` / `Enable ordering`). The pause/unpause control is hidden entirely while the vendor is menu-only, since pausing intake that is already off is meaningless. No tooltips, no info icons, no warning boxes.

Pod detail (`AdminPodOverview.tsx`) — the same `Ordering mode` section for pod-wide intent. The vendor roster shows each vendor's *effective* state (`Orderable`, `Menu only`, `Menu only — pod`

`disabled`, `Ordering setup incomplete`) and offers a per-vendor ordering control inside the existing collapsed "Roster management" area. The pod-wide toggle is not duplicated per row. A `Menu only` filter tab appears only once at least one vendor is menu-only, to keep the tab row short.

Vendor list (`AdminVendorSearchForm.tsx`, `vendors/page.tsx`) — a new `Ordering` filter (`Orderable` / `Menu only`), separate from the existing routing filter.

Audit — `VENDOR_ORDERING_MODE_MENU_ONLY`, `VENDOR_ORDERING_MODE_ENABLED`, `POD_ORDERING_MODE_MENU_ONLY`, `POD_ORDERING_MODE_ENABLED`, each recording old and new intent plus the admin reason.

Authorization — both actions go through `withAdmin`. Pod owners have no path to `orderingEnabled` in Stage 1.

Customer UI

Fully menu-only pod

The pod-wide ticker returns a neutral `Browse menus` status (`tone: "menu_only"`) rather than "Not accepting orders". The hero drops the "One cart · One pickup" badge and uses a browse-oriented tagline. Group-order CTAs are replaced with a single `Browse menus` button, and the QR-entry banner uses browse wording. The vendor section header reads "Browse menus at {pod}" instead of "Order from vendors at {pod}".

Mixed pod

Every vendor stays visible. Orderable vendors keep the normal `Order now` CTA; menu-only vendors get `View menu`. There is no disabled order button anywhere. The `Menu only` badge appears on cards **only in mixed pods** — in a fully menu-only pod the label would be noise on every card, and the pod-level status already says it. Menu-only vendors are excluded from the open/closed counts, so eight menu-only vendors alongside two open ones reads as "Open for orders", not "2 of 10 vendors open".

Vendor menu page

Branding, description, hours, categories, prices, item descriptions, images, and sold-out state all render normally. Add-to-cart and quantity controls are *removed*, not disabled, and the card body is no longer an activatable button, so the customization modal cannot open. The status appears once near the hero as `Open · Menu only` or `Closed · Menu only`; it is never repeated per item.

The critical separation in `VendorMenuItemCard`:

```
const itemUnavailable = !item.isAvailable;                              // sold out, data only
const dimmed = menuOnly ? itemUnavailable : orderingDisabled || itemUnavailable;
const interactive = !menuOnly && !orderingDisabled && !itemUnavailable;
```

An available item in a menu-only vendor looks completely normal. A sold-out item still looks sold out.

Group Ordering

`hasOrderableVendor` is computed in `pod-customer-page-data.ts` and gates the start/join affordances on both the standard and destination pod templates, plus `shouldOfferDestinationGroupOrderPrompt`.

Menu-only vendors cannot enter a group order because group carts use the same `addCartItem` path, which rejects them. Mixed pods therefore keep group ordering for the eligible vendors with no additional work.

An already-active group cart keeps its `Open group cart` CTA even in a fully menu-only pod. Only the start and join affordances go away — nobody loses access to in-flight work.

Cart / Checkout

Two new codes, distinct from pause, closed, item-unavailable, and Stripe-incomplete:

```
POD_ORDERING_DISABLED
VENDOR_ORDERING_DISABLED
```

Enforcement points:

Path	Enforcement
`addCartItem`	Intent passed into `getVendorOrderabilityInPod` (full readiness path)
`updateCartItem`	`assertOrderingIntentAllowsCartLine` — shallow path, reads intent from the DB
`validateCartItemsForDisplay`	Per-line intent check from `prisma.vendor.findMany`, before all other checks
`validateCartForOrder`	Per-line intent check from `prisma.vendor.findMany` **(fixed this stage)**
`removeCartItem`, `updateCartItem(quantity: 0)`	Deliberately **not** gated, so blocked lines can always be cleared

Existing carts are never silently emptied. Blocked lines get action-oriented copy — "This vendor is currently menu-only and is not accepting Open Order orders. Remove these items to continue." — and lines from other, still-orderable vendors stay valid, so a mixed cart can be checked out after removing the blocked ones.

Server-side rejection does not depend on UI hiding anywhere.

Readiness / Stripe / Routing

The readiness model now has two clearly separate questions.

Menu readiness (can the vendor appear publicly?) — unchanged inputs: active vendor, active pod membership, active pod, required public profile, published menu, hours. Stripe, POS, routing, and ordering intent are not inputs.

Ordering readiness (can the vendor accept paid orders?) — menu readiness **plus** vendor intent, pod intent, payment readiness, routing readiness, not paused, currently open, and at least one available item.

Consequences of that split:

- `getVendorOperationalMissingItems` suppresses commerce-prerequisite gaps while menu-only, so they never surface as failed checklist items.
- `buildSetupChecklist` filters out `stripe`, `pos`, and `menu_available` for menu-only vendors, and `VENDOR_MENU_ONLY_SETUP_REQUIRED_CHECKLIST_KEYS` defines what "complete" means for them.
- `deriveVendorAttentionItems` no longer raises "ordering closed" for a menu-only vendor.
- Routing readiness warnings, POS setup prompts, and kitchen-mode prompts are hidden.
- No integration configuration is written, deleted, or switched. Menu-source ownership reconciliation is never invoked by a mode change. No Stripe account is created when ordering is turned back on.

Vendor Dashboard

`src/lib/vendor-dashboard-nav-mode.ts` (new) owns the visibility rules; `src/lib/vendor-dashboard-ordering-mode.server.ts` (new) loads intent plus active-order and order-history counts once per request and injects them through the vendor layout.

Hidden when menu-only: Payouts, Kitchen, POS/integrations setup, routing setup, active-order operations, the GMV/today-performance section, and payment-setup banners.

Kept when menu-only: Dashboard, Menu, Hours, Vendor Profile, Setup.

Two guards protect work the vendor still owes:

- **Active ticket** — if any order is new/preparing/ready, Kitchen and the active-orders section stay visible. Ordering being switched off never hides a ticket that still needs fulfilling.
- **Order history** — if the vendor has ever had an order, the Orders route stays reachable so history and refunds are not stranded.

Turning ordering back on restores the full navigation automatically; nothing is cached or persisted per-vendor.

Dashboard home leads with `Your menu is live` and menu status instead of GMV. "No orders today" is not shown as a negative state. Quick links drop Payouts and re-word Menu and Hours around browsing. `Store status` becomes `Menu status`, and the Payments and Routing tiles are removed.

Hours copy changes from "Customer ordering hours" to "Hours" when menu-only. The stored field and schema are unchanged.

Payouts page: description acknowledges nothing there is required, and the "finish payment setup" prompt is suppressed. Existing Stripe connections are preserved and the page remains reachable.

Pod Dashboard

A fully menu-only pod no longer reads as unhealthy.

- `PodStatusCard` shows `Listed vendors` instead of `Orderable vendors`, adds a single `Menu only` badge, and explains the state in one line.
- `PodTodayActivitySection` replaces orders/sales/GMV metrics with `Vendors in pod` and `Menus live`, plus a link to the QR and sharing tools.
- `PodVendorReadinessSection` becomes "Vendor menus" and labels menu-only vendors as `Menu only`, not as blocked.
- `derivePodAttentionItems` no longer raises "No vendors are currently orderable" when that is the configured outcome, and stops chasing Stripe entirely. Hours and menu gaps still surface, because those gate *public visibility* too — only the stated consequence changes.
- `derivePodSetupChecklist` asks for "At least one listed vendor" instead of "At least one orderable vendor".
- `computePodLaunchReadinessSummary` counts menu-only vendors separately and describes them accurately.

For mixed pods, commerce metrics stay. Menu-only vendors are never counted as failed orderable vendors, and listed/menu-ready is reported separately from orderable.

Pod owners cannot change `orderingEnabled`. The admin remains the control point.

Existing Orders

Disabling ordering blocks **new** orders only. No order query filters on `orderingEnabled` — active orders, VendorOrders, historical orders, refunds, payouts, and routing history are all untouched and remain visible. A vendor with an in-flight ticket keeps the interface needed to finish it.

Tests

95 new passing tests. Baseline before this work: 2,947 passing / 32 failing across 19 files. After: **3,042 passing / 32 failing across the same 19 files** — no new failures, and the pre-existing failures are unrelated (Square OAuth debug routes, payout guardrails, Deliverect menu-route gating, `order.service.pending-reuse`). Type errors held at the pre-existing 81.

Added

File	Coverage

`src/lib/vendor-ordering-mode.test.ts` (25)	Intent resolution incl. undefined-as-enabled; all four pod/vendor flag combinations; setup-incomplete when intent is on but Stripe is missing; pause / closed / sold-out each stay distinct; menu-only vendor stays publicly visible with no banner; validation codes and their precedence over pause; readiness **and** shallow orderability paths; cart copy per reason; admin label keys
`src/lib/vendor-dashboard-nav-mode.test.ts` (6)	Full nav when enabled; Payouts/Kitchen hidden when menu-only; Menu/Hours/Profile/Setup kept; Kitchen and Orders retained for an in-flight ticket; history reachable after ordering is off; nav restored on re-enable
`src/lib/menu-only-surfaces.test.ts` (25)	Migration additivity, `DEFAULT true`, no backfill, no menu/routing/payment columns touched; both admin actions behind `withAdmin`; pod owners have no `orderingEnabled` path; admin controls and list filter present; customer browse CTA and single hero status; neutral pod status; group-CTA gating; vendor dashboard nav/setup/hours/payouts adaptations; pod dashboard listed-vendor metrics and attention suppression; order queries unfiltered by intent; mode services free of menu/routing/payment writes
`src/services/admin-ordering-mode.service.test.ts` (10)	Vendor and pod mode changes write **only** `orderingEnabled`; explicit assertions that `menuSource`, `orderRoutingMode`, `mennyuOrdersPaused`, `isActive`, Stripe fields, Square config, and Deliverect config are absent from the write; `MenuItem` never updated; vendor intent survives a pod-wide switch; audit entries with old/new intent; reason required; no-op rejected; re-enable does not republish
`src/services/cart.service.menu-only.test.ts` (5)	`updateCartItem` rejects for menu-only vendor and menu-only pod with the correct code and copy; rejection is a `CartValidationError` bound to the line; removal via `quantity: 0` and `removeCartItem` still succeed

Updated

File	Change
`src/services/order.service.cart-validation.test.ts` (+5)	Checkout rejects menu-only vendor and menu-only pod; menu-only wins over pause; mixed cart flags only the menu-only vendor's line and leaves the other vendor valid; a menu-only pod flags every line
`src/lib/pod-page-status.test.ts` (+3)	All-menu-only pod returns `Browse menus`; mixed pod counts only ordering-intent vendors; menu-only vendors cannot make a mixed pod read as closed
`src/components/vendor-menu/vendor-menu-item-card.test.ts` (+3)	Sold-out separated from menu-only; controls removed rather than disabled; sold-out badge still renders for a menu-only unavailable item. The existing accessible-button assertion was retightened to the new `interactive` guard, which now also covers temporarily-blocked ordering
`src/lib/pod-readiness-page.test.ts`	Fixture carries explicit `menuOnly`

`src/lib/admin-pod-summary.test.ts`, `src/lib/admin-vendor-summary.test.ts`	Fixtures carry `orderingEnabled`

Files Changed

New

```
prisma/migrations/20260903230000_pod_vendor_ordering_enabled/migration.sql
src/lib/vendor-ordering-mode.ts
src/lib/vendor-dashboard-nav-mode.ts
src/lib/vendor-dashboard-ordering-mode.server.ts
src/lib/vendor-ordering-mode.test.ts
src/lib/vendor-dashboard-nav-mode.test.ts
src/lib/menu-only-surfaces.test.ts
src/services/admin-ordering-mode.service.test.ts
src/services/cart.service.menu-only.test.ts
```

Modified

Schema and central state:

```
prisma/schema.prisma
src/lib/vendor-readiness-states.ts
src/lib/vendor-readiness-validation.server.ts
src/lib/vendor-orderability-in-pod.ts
src/lib/pod-route-resolve.ts
```

Cart and checkout:

```
src/services/cart.service.ts
src/services/order.service.ts
```

Admin:

```
src/lib/admin-audit-log.ts
src/lib/admin-pod-summary.ts
src/lib/admin-vendor-summary.ts
src/actions/admin-pod.actions.ts
src/actions/admin-vendor.actions.ts
src/services/admin-pod-rescue.service.ts
src/services/admin-vendor-rescue.service.ts
src/services/admin-pod-detail.service.ts
src/services/admin-vendor-detail.service.ts
```

```
src/app/admin/(dashboard)/pods/[podId]/AdminPodOverview.tsx
src/app/admin/(dashboard)/vendors/[vendorId]/AdminVendorOverview.tsx
src/app/admin/(dashboard)/vendors/AdminVendorSearchForm.tsx
src/app/admin/(dashboard)/vendors/page.tsx
```

Customer:

```
src/lib/pod-customer-page-data.ts
src/lib/pod-page-status.ts
src/lib/destination-pod-group-prompt.ts
src/lib/vendor-menu-customer-page-render.tsx
src/components/pod/PodPageHero.tsx
src/components/pod/PodPageHeroActions.tsx
src/components/pod/PodPageVendorSection.tsx
src/components/pod/PodVendorCard.tsx
src/components/pod/PodVendorGrid.tsx
src/components/pod/StandardPodPageView.tsx
src/components/pod/destination/DestinationPodPageView.tsx
src/components/pod/destination/DestinationPodVendorCard.tsx
src/components/pod/destination/DestinationPodVendorSection.tsx
src/components/vendor-menu/VendorMenuItemCard.tsx
src/components/vendor-menu/VendorMenuExperience.tsx
src/components/vendor-menu/VendorMenuExperienceClient.tsx
src/components/vendor-menu/VendorMenuHero.tsx
```

Vendor dashboard:

```
src/lib/vendor-dashboard-data.server.ts
src/lib/vendor-dashboard-attention.ts
src/lib/vendor-pod-readiness.ts
src/lib/vendor-operational-copy.ts
src/app/vendor/[vendorId]/layout.tsx
src/app/vendor/[vendorId]/VendorLayoutChrome.tsx
src/app/vendor/[vendorId]/VendorAreaNav.tsx
src/app/vendor/[vendorId]/dashboard/page.tsx
src/app/vendor/[vendorId]/dashboard/VendorStoreStatusCard.tsx
src/app/vendor/[vendorId]/dashboard/VendorQuickLinksSection.tsx
src/app/vendor/[vendorId]/setup/page.tsx
src/app/vendor/[vendorId]/hours/page.tsx
src/app/vendor/[vendorId]/hours/VendorCustomerOrderingHoursForm.tsx
src/app/vendor/[vendorId]/payouts/page.tsx
```

Pod dashboard:

```
src/lib/pod-dashboard-data.server.ts
src/lib/pod-dashboard-attention.ts
```

```
src/lib/pod-vendor-adoption.ts
src/lib/pod-readiness-page.ts
src/app/pod/[podId]/dashboard/page.tsx
src/app/pod/[podId]/dashboard/PodStatusCard.tsx
src/app/pod/[podId]/dashboard/PodTodayActivitySection.tsx
src/app/pod/[podId]/dashboard/PodVendorReadinessSection.tsx
src/app/pod/[podId]/dashboard/PodVendorRosterPanel.tsx
src/app/pod/[podId]/dashboard/PodRosterReadinessSummary.tsx
```

Tests:

```
src/services/order.service.cart-validation.test.ts
src/lib/pod-page-status.test.ts
src/lib/pod-readiness-page.test.ts
src/lib/admin-pod-summary.test.ts
src/lib/admin-vendor-summary.test.ts
src/components/vendor-menu/vendor-menu-item-card.test.ts
```

> *The working tree also carries unrelated in-progress changes to menu-source ownership (`src/lib/vendor-menu-source.ts`, `src/services/vendor-menu-source-ownership*`, `src/services/menu-apply-canonical-live.ts`, `src/services/native-open-order-availability-repair*`, `src/services/vendor-menu-catalog-adoption*`, `src/services/menu-publish-from-canonical.service.ts`, `scripts/repair-vendor-menu-source-ownership.ts`). Those predate this stage and are not part of it.*

Deferred Work

Explicitly out of scope for Stage 1 and not implemented:

- **Concierge vendor creation / vendor claiming** — no unclaimed-vendor model, no admin create-unclaimed-vendor flow, no claim invitation. A menu-only vendor still requires a claimed account today.
- **Engagement analytics** — no new events, no warehouse changes, no menu-view or browse tracking. Menu-only pods currently have commerce analytics de-emphasized but nothing measuring engagement in their place.
- **Pod-owner ordering controls** — pod owners can see effective state but cannot change `orderingEnabled` for their pod or their vendors.
- **Self-service menu creation expansion** — the menu builder is unchanged.
- **New POS integrations**, and no removal of existing routing integrations.
- **Pod dashboard navigation** — the pod owner's Payouts and Analytics routes are still present for a fully menu-only pod. The dashboard cards no longer lead with commerce, but the routes themselves were left alone to keep this stage's diff contained. Worth revisiting.
- **Admin bulk mode changes** — switching many vendors is one action per vendor. Acceptable for the current roster sizes.

Risks / Follow-Up

Needs manual QA — the following were verified by unit and contract tests but not against a running app or real data:

1. **Fully menu-only pod, end to end.** Pod page loads publicly, vendor cards render normally, menus and prices show, sold-out state still works, no add-to-cart controls anywhere, no group-order CTA, no payment or POS failure messaging.
2. **Mixed pod at realistic scale** (e.g. 8 menu-only + 2 orderable). All 10 visible, 2 with an ordering CTA, 8 with `View menu`, cart accepts only the 2, and the pod status does not imply the 8 are broken.
3. **Pod-wide toggle round trip.** Disable pod ordering, confirm all vendors are effectively menu-only and their own flags are unchanged in the database, re-enable, confirm previously orderable vendors resume with no menu republish.
4. **Integration preservation.** For a vendor with real Square, Deliverect, and Stripe configuration: disable ordering, confirm the configuration rows are intact, re-enable, confirm ordering resumes against the same configuration. The tests assert the service never writes those fields, but this should be confirmed against live data.
5. **Active order during a mode flip.** With a live ticket, switch the vendor to menu-only, confirm Kitchen and the active-orders section remain reachable and the ticket can be completed, and confirm no new order can be created.
6. **Stale cart in a real browser.** Add items, flip the vendor to menu-only in another tab, then try to change quantity and to check out. Both should fail with the menu-only message, and removing the line should work.

Known residual risks:

- **Deliverect hours sync.** A menu-only vendor whose hours sync from Deliverect will still show `Closed · Menu only` outside those hours. That is defensible, but if a menu-only vendor's Deliverect channel goes quiet, the public menu page may read as closed indefinitely. Worth a look during QA.
- **Copy audit on secondary surfaces.** The primary customer, vendor, and pod surfaces were reworded, but transactional emails, receipts, and less-trafficked admin screens were not swept for "ordering" wording that a menu-only pod would never trigger. Low impact, since those paths require an order to exist.
- **Analytics blind spot.** With commerce metrics de-emphasized and no engagement metrics yet, a fully menu-only pod's dashboard is quiet by design. Pod owners may read that as "nothing is happening" until the deferred engagement analytics land.